

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME DE GUIA 02

“ALGORITMOS DE APROXIMACIÓN APLICADOS A SCHEDULING: LIST SCHEDULING Y LONGEST PROCESSING TIME FIRST”

Asignatura: ALGORITMOS AVANZADOS

Docente: Ing. Héctor Eduardo Ugarte Rojas

ID: GRUPO 5

Presentado Por:

- | | |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Porroa Sivana, Yeni Ruth | 120893 |
| • Quispe Rimachi, Romario | 164257 |
| • Yaranga Achahui, Aldo | 103179 |

Cusco-Perú -2026

Índice

1. Instrucciones de ejecución	1
2. Respuestas del trabajo preparatorio	1
3. Implementación y complejidad	2
3.1. Representación y cota inferior	2
3.2. List Scheduling y optimización del Heap	3
3.3. Ramificar-Podar exacto	4
4. Pruebas de factibilidad y casos límite	4
4.1. Instancias ajustadas de Graham	4
5. Generador y conjunto reproducible de instancias	5
6. Propuesto 1: evaluación de calidad frente al óptimo	5
6.1. Análisis de dominancia: ¿LPT domina siempre a List Scheduling?	7
6.2. Rendimiento y límite práctico del método exacto	8
7. Propuesto 2: efecto del orden y escalabilidad	8
7.1. Distinción metodológica: C_{ALG}/LB frente a C_{ALG}/OPT	8
7.2. Sensibilidad al orden y la cota $p_{\text{máx}}/LB$	9
7.3. Costo computacional del ordenamiento vs. mejora del makespan	10
8. Respuestas a las preguntas de análisis	10
9. Conclusiones grupales	13

Índice de tablas

1.	Organización del notebook de experimentación en Google Colab	1
2.	Comparación de tiempo de ejecución en List Scheduling ($m = 4$)	4
3.	Resumen de la suite de pruebas unitarias y validación de casos límite	5
4.	Comportamiento de las instancias ajustadas de Graham	5
5.	Familias distribucionales empleadas en la experimentación	6
6.	Rendimiento de calidad en las 72 instancias aleatorias del Propuesto 1	6
7.	Instancias de excepción donde List Scheduling superó a LPT	7
8.	Exploración exhaustiva de órdenes en instancias ajustadas de LPT	8
9.	Escalabilidad de Ramificar-Podar con $m = 4$ (familia estrecha)	8
10.	Makespan y razones relativas a la cota inferior en instancias a gran escala	9
11.	Sensibilidad de List Scheduling ante 1 000 permutaciones aleatorias ($n = 100$)	10
12.	Costo computacional del ordenamiento frente a la reducción del makespan	11

Índice de figuras

1.	Distribución de razones observadas $r(I)$ frente a las cotas teóricas de Graham	7
2.	Exploración de nodos y linealidad estricta entre tiempo y nodos en Ramificar-Podar	9
3.	Distribución de makespan en 1 000 órdenes aleatorios para $n = 100, m = 64$	10
4.	Convergencia asintótica del exceso sobre la cota inferior en función de n	11
5.	Tiempos de ejecución y descomposición del costo de ordenamiento en escala log-log	11

1. Instrucciones de ejecución

La totalidad de los códigos resueltos, propuestos y scripts de medición se encuentran organizados en el notebook Laboratorio02_Colab.ipynb, estructurado según la Tabla 1.

Tabla 1

Organización del notebook de experimentación en Google Colab

Sección	Contenido programático
0	Inicialización, definición de la semilla maestra ($SEMILLA_GLOBAL = 2026$) y cronometraje.
1 a 3	Ejercicios resueltos: representación, List Scheduling, LPT y Ramificar-Podar.
4	Suite de pruebas de factibilidad y análisis de casos límite (36 pruebas exhaustivas).
5	Análisis empírico de complejidad: versión de la guía frente a versión optimizada de LS.
6	Generador reproducible de instancias por familias distribucionales.
7	Propuesto 1: Calidad frente al óptimo, métricas relativas y límite del método exacto.
8	Propuesto 2: Efecto del orden, sensibilidad según razón n/m y escalabilidad asintótica.
9	Exportación automática de resultados (archivos CSV, instancias JSON y gráficos).

Pasos de ejecución:

1. Cargar el archivo Laboratorio02_Colab.ipynb en Google Colab.
2. Ejecutar el notebook completo mediante **Entorno de ejecución** → **Ejecutar todas** (Ctrl + F9). La duración total oscila entre 30 y 45 segundos.
3. La celda final empaqueta y descarga un archivo comprimido (resultados_lab02.zip) con todos los registros experimentales y figuras generadas.

2. Respuestas del trabajo preparatorio

a) ¿Por qué List Scheduling es un algoritmo de aproximación y no solo una heurística?

Una heurística es una regla empírica que suele funcionar bien pero carece de garantías analíticas sobre su peor caso. List Scheduling es un **algoritmo de aproximación** formal porque satisface dos propiedades teóricas estrictas:

- Ejecuta en tiempo polinomial ($O(n \log m)$).
- Posee una garantía de aproximación demostrada para **toda** instancia posible I :

$$C_{LS}(I) \leq \left(2 - \frac{1}{m}\right) \text{OPT}(I). \quad (1)$$

Demostración breve: Sea j el trabajo que finaliza al último determinando el makespan ($C_{LS} = S_j + p_j$). Al momento de su asignación, la máquina receptora era la de mínima

carga acumulada, por lo que su instante de inicio S_j no supera la carga promedio de los demás trabajos: $S_j \leq \frac{1}{m} \sum_{k \neq j} p_k = \frac{1}{m} (\sum_k p_k - p_j)$. Sustituyendo:

$$C_{LS} \leq \frac{\sum_k p_k}{m} + \left(1 - \frac{1}{m}\right) p_j \leq \text{OPT} + \left(1 - \frac{1}{m}\right) \text{OPT} = \left(2 - \frac{1}{m}\right) \text{OPT}. \quad (2)$$

b) ¿Cuál es la diferencia entre una garantía ρ y una razón observada $r(I)$?

La garantía ρ es un **teorema analítico** que acota el peor caso en todo el universo de instancias ($\forall I : C(I) \leq \rho \cdot \text{OPT}(I)$). La razón observada $r(I) = \text{ALG}(I)/\text{OPT}(I)$ es una **medición empírica** sobre una instancia concreta I . Siempre se cumple $1 \leq r(I) \leq \rho$. Un experimento puede desmentir una cota teórica si encuentra $r(I) > \rho$ (evidenciando un error de implementación o de cota), pero observar $r(I) \leq \rho$ en muchas instancias jamás demuestra el teorema general.

c) ¿Por qué alcanzar una cota inferior válida demuestra optimalidad?

Por definición de cota inferior válida, ninguna solución factible puede tener makespan menor que LB , es decir, $LB \leq \text{OPT}$. Si una solución factible S alcanza $C(S) = LB$, por transitividad:

$$LB \leq \text{OPT} \leq C(S) = LB \implies C(S) = \text{OPT}. \quad (3)$$

La planificación S queda formalmente certificada como óptima sin necesidad de buscar otras alternativas.

d) ¿Por qué LPT procesa primero los trabajos de mayor duración?

Porque en List Scheduling el exceso respecto al promedio ideal proviene del último trabajo asignado: $C \leq \frac{\sum p}{m} + (1 - 1/m)p_{\text{último}}$. Al ordenar los trabajos de forma decreciente, los trabajos que cierran la planificación son los más pequeños. Graham demostró que si el último trabajo mide más de $\text{OPT}/3$, cada máquina del óptimo contiene a lo sumo dos trabajos (donde LPT resulta óptimo); si mide a lo sumo $\text{OPT}/3$, el residuo queda acotado por dicho valor, reduciendo la garantía teórica de $2 - 1/m$ a:

$$C_{LPT} \leq \left(\frac{4}{3} - \frac{1}{3m}\right) \text{OPT}. \quad (4)$$

3. Implementación y complejidad

3.1. Representación y cota inferior

Una instancia está formada por una lista de tiempos enteros $p_j \in \mathbb{Z}^+$ y un número de máquinas $m \geq 1$. La planificación se modela como una lista de m sublistas. La validación de factibilidad emplea `collections.Counter` para garantizar que las multiplicidades de trabajos

repetidos se preserven exactamente, evitando las omisiones que ocurrirían con conjuntos (set). La cota inferior se calcula en aritmética entera:

$$LB = \max \left(\left\lfloor \frac{\sum p_j + m - 1}{m} \right\rfloor, \max_j p_j \right) \leq \text{OPT}. \quad (5)$$

3.2. List Scheduling y optimización del Heap

En la implementación preliminar sugerida por la guía de laboratorio, cada nodo del heap almacenaba la terna (carga, id, asignados) y generaba una copia por concatenación en cada paso (asignados + [trabajo]). Para una máquina que recibe $k = n/m$ trabajos, el costo acumulado de copiado es $\sum_{i=1}^{n/m} O(i) = O(n^2/m^2)$, lo que eleva el tiempo total a $O(n \log m + n^2/m)$, degradando severamente el algoritmo a comportamiento cuadrático para valores moderados de n .

Se rediseñó la estructura desacoplando las listas de trabajos del heap. El heap almacena únicamente tuplas de tamaño fijo (carga, id_maquina), permitiendo inserciones en tiempo $O(1)$ mediante `planificacion[id].append(trabajo)`:

```

1 import heapq
2
3 def list_scheduling(trabajos, m):
4     if m <= 0:
5         raise ValueError("El numero de maquinas debe ser positivo")
6     planificacion = [[] for _ in range(m)]
7     heap = [(0, i) for i in range(m)]
8     heapq.heapify(heap)
9
10    for trabajo in trabajos:
11        carga, maquina = heap[0] # O(1): maquina menos
12        # cargada
13        planificacion[maquina].append(trabajo) # O(1): insercion sin
14        # copia
15        heapq.heapreplace(heap, (carga + trabajo, maquina)) # O(log m)
16
17    return planificacion, max(carga for carga, _ in heap)
18
19 def lpt(trabajos, m):
20     ordenados = sorted(trabajos, reverse=True) # O(n log n)
21     return list_scheduling(ordenados, m) # O(n log m)

```

La Tabla 2 expone la evaluación empírica de escalabilidad con $m = 4$. Mientras la versión de la guía cuadruplica su tiempo cada vez que n se duplica (crecimiento cuadrático $\times 16$), la versión optimizada mantiene un factor de escala estrictamente lineal ($\times 4$), logrando una aceleración de 141.8 veces para $n = 64000$.

Tabla 2*Comparación de tiempo de ejecución en List Scheduling ($m = 4$)*

n	Versión optimizada	Versión de la guía	Cociente (Guía / Opt.)	Factor de escala
1 000	202.0 μ s	813.6 μ s	4.0	—
4 000	776.6 μ s	8.47 ms	10.9	$\times 3,8 / \times 10,4$
16 000	3.12 ms	114.90 ms	36.8	$\times 4,0 / \times 13,6$
64 000	12.33 ms	1.749 s	141.8	$\times 4,0 / \times 15,2$

3.3. Ramificar-Podar exacto

El método exacto explora el árbol de decisiones colocando en el nivel k el k -ésimo trabajo más largo. Incorpora:

1. **Incumbente inicial eficiente:** Inicializado con la solución generada por LPT.
2. **Certificación inmediata en la raíz:** Si el valor retornado por LPT iguala a LB , se termina con 0 nodos explorados.
3. **Rompimiento de simetrías:** Si múltiples máquinas poseen exactamente la misma carga acumulada, o están vacías, se explora únicamente una de ellas.
4. **Poda por cota dinámica:** Se poda toda rama cuya cota inferior local supere o iguale al incumbente:

$$LB_{\text{nodo}} = \max \left(\max_i L_i, \left\lceil \frac{\sum L_i + \sum_{j>k} P_j}{m} \right\rceil \right) \geq \text{mejor_valor}. \quad (6)$$

4. Pruebas de factibilidad y casos límite

Se diseñó una suite de 36 pruebas unitarias automáticas para certificar la corrección del sistema. La totalidad de las pruebas fueron superadas satisfactoriamente (Tabla 3).

4.1. Instancias ajustadas de Graham

Para validar las cotas teóricas de peor caso se construyeron las familias ajustadas analíticas:

- **Peor caso de List Scheduling:** Instancia compuesta por $m(m-1)$ trabajos de duración 1 seguidos de un único trabajo de duración m . Las máquinas se cargan equitativamente a nivel $m-1$ y el trabajo largo recae al final: $C_{LS} = 2m-1$ mientras $OPT = m$, alcanzando la razón exacta $(2m-1)/m = 2 - 1/m$.

Tabla 3*Resumen de la suite de pruebas unitarias y validación de casos límite*

Categoría	Casos de prueba evaluados	Resultado observado
Factibilidad estricta	Planificaciones válidas, trabajos faltantes, trabajos duplicados y número incorrecto de máquinas.	Counter discrimina anomalías con exactitud.
Validación de dominio	Entradas erróneas ($m \leq 0$, $p_j \leq 0$, flotantes).	Disparo de ValueError.
Casos degenerados	Sin trabajos ($n = 0$), una máquina ($m = 1$), más máquinas que trabajos ($m \geq n$).	Makespans coinciden con las sumas y máximos teóricos.
Cota inalcanzable	Instancia $[2, 2, 2]$ con $m = 2$: $LB = 3$, pero $OPT = 4$.	El algoritmo exacto demuestra $OPT = 4$.
Batería masiva	300 instancias aleatorias (900 ejecuciones entre LS, LPT y B&B).	100% factibles y satisfacen $OPT \leq \min(C_{LS}, C_{LPT})$.

- **Peor caso de LPT:** Dos trabajos de cada tamaño $2m - 1, 2m - 2, \dots, m + 1$ y tres trabajos de tamaño m . LPT obtiene $4m - 1$ frente a un óptimo de $3m$, igualando exactamente la razón $(4m - 1)/(3m) = 4/3 - 1/(3m)$.

Como resume la Tabla 4, las razones observadas coinciden con precisión analítica con las cotas teóricas, demostrando que ninguna de las cotas puede ser reducida algebraicamente.

Tabla 4*Comportamiento de las instancias ajustadas de Graham*

m	C_{LS}/OPT obs.	Garantía LS ($2 - 1/m$)	C_{LPT}/OPT obs.	Garantía LPT ($4/3 - 1/(3m)$)
2	$3/2 = 1,5000$	1.5000	$7/6 = 1,1667$	1.1667
3	$5/3 = 1,6667$	1.6667	$11/9 = 1,2222$	1.2222
4	$7/4 = 1,7500$	1.7500	$15/12 = 1,2500$	1.2500

5. Generador y conjunto reproducible de instancias

Para aislar los experimentos de variaciones dependientes de plataforma, cada instancia se derivó mediante una semilla unívoca calculada como $semilla_k = SEMILLA_GLOBAL \cdot 1000 + k$, persistiendo todas las especificaciones en `instancias.json`.

6. Propuesto 1: evaluación de calidad frente al óptimo

Se generaron 72 instancias combinando $m \in \{2, 3, 4\}$, $n \in \{6, 9, 12, 15\}$ y las tres primeras familias distribucionales, más las 6 instancias ajustadas analíticas (total: 78 instancias).

Tabla 5*Familias distribucionales empleadas en la experimentación*

Familia	Distribución estocástica	Propósito analítico
uniforme	$p_j \sim U[1, 20]$	Variabilidad estándar balanceada; escenario típico.
estrecha	$p_j \sim U[10, 20]$	Duraciones casi homogéneas; reduce la eficacia de poda y desafía el ordenamiento de LPT.
sesgada	75 % $\sim U[1, 5]$, 25 % $\sim U[15, 30]$	Presencia de trabajos desproporcionados; castiga a List Scheduling si arriban al cierre.
grande	$p_j \sim U[1, 100]$	Instancias de alta escala para evaluación asintótica.

Tabla 6*Rendimiento de calidad en las 72 instancias aleatorias del Propuesto 1*

Grupo	Algoritmo	Óptimos	r prom.	r med.	r máx.	δ prom. (%)	Garantía ρ
$m = 2$ (24)	LS	6 (25.0 %)	1.0690	1.0426	1.3333	6.90 %	1.500
	LPT	17 (70.8 %)	1.0116	1.0000	1.0735	1.16 %	1.167
$m = 3$ (24)	LS	1 (4.2 %)	1.1289	1.0808	1.3333	12.89 %	1.667
	LPT	19 (79.2 %)	1.0052	1.0000	1.0488	0.52 %	1.222
$m = 4$ (24)	LS	3 (12.5 %)	1.1158	1.1013	1.3529	11.58 %	1.750
	LPT	17 (70.8 %)	1.0188	1.0000	1.1212	1.88 %	1.250
Global (72)	LS	10 (13.9 %)	1.1046	1.0613	1.3529	10.46 %	1.500 a 1.750
	LPT	53 (73.6 %)	1.0119	1.0000	1.1212	1.19 %	1.167 a 1.250

Hallazgos clave:

- **Cero violaciones teóricas:** El 100 % de las soluciones verificó $m \cdot C_{LS} \leq (2m - 1)OPT$ y $3m \cdot C_{LPT} \leq (4m - 1)OPT$.
- **Cercanía al óptimo:** La mediana de r_{LPT} fue 1,0000, con una desviación promedio de solo 1,19 %, confirmando que la garantía de Graham constituye una cota conservadora de peor caso.
- **Efecto de la distribución:** LPT alcanzó el óptimo en el 100 % de las instancias sesgadas, pero redujo su efectividad al 58.3 % en la familia estrecha.

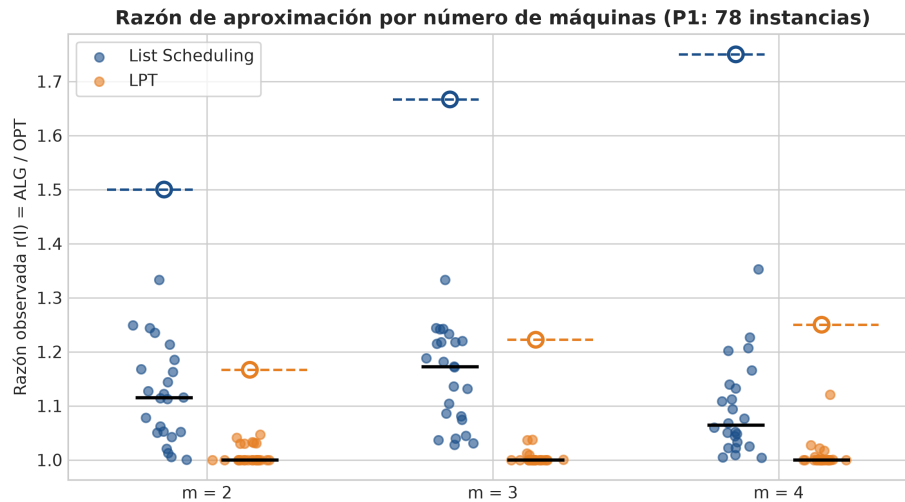


Figura 1
Distribución de razones observadas $r(I)$ frente a las cotas teóricas de Graham

6.1. Análisis de dominancia: ¿LPT domina siempre a List Scheduling?

En la comparación directa sobre las 72 instancias, LPT superó a List Scheduling en 53 casos y empató en 13. No obstante, se registraron **6 excepciones** donde List Scheduling con el orden generado superó a LPT (Tabla 7).

Tabla 7
Instancias de excepción donde List Scheduling superó a LPT

Instancia	Familia	n	m	OPT	C_{LS}	C_{LPT}
P1-003	uniforme	9	2	47	47	48
P1-014	uniforme	12	3	41	42	43
P1-027	estrecha	9	2	68	71	73
P1-028	estrecha	9	2	75	77	79
P1-031	estrecha	15	2	111	114	115
P1-043	estrecha	9	4	33	36	37

Justificación teórica: Para cualquier problema $P_m \parallel C_{\max}$ existe al menos una permutación de trabajos con la cual List Scheduling produce la solución óptima (basta ingresar los trabajos agrupados por máquina según la asignación óptima). LPT no busca dicha permutación; fija rígidamente un orden decreciente. Al explorar el espacio completo de permutaciones sobre las instancias ajustadas (Tabla 8), se observa que en $m = 3$ más del 54 % de las permutaciones superan a LPT. Por tanto, **LPT no domina estrictamente a List Scheduling en cada instancia**, aunque sí domina de forma categórica en su garantía analítica de peor caso.

Tabla 8*Exploración exhaustiva de órdenes en instancias ajustadas de LPT*

m	Conjunto de trabajos	Total permutaciones	OPT	C_{LPT}	Órdenes con $C_{LS} < C_{LPT}$
2	[3, 3, 2, 2, 2]	120	6	7	24 (20.0 %)
3	[5, 5, 4, 4, 3, 3, 3]	5 040	9	11	2 736 (54.3 %)

6.2. Rendimiento y límite práctico del método exacto

Se evaluó el comportamiento de Ramificar-Podar en la familia estrecha ($m = 4$) incrementando n desde 10 hasta 24 con un tope de seguridad de 10^6 nodos. Los resultados revelan una dicotomía fundamental (Tabla 9):

- **Casos triviales (n múltiplo de m):** Para $n \in \{12, 16, 20, 24\}$, LPT equilibra los trabajos de forma exacta alcanzando $\lceil \sum p_j / m \rceil = LB$, concluyendo con **0 nodos** explorados.
- **Casos no homogéneos (n no divisible por m):** El algoritmo debe probar combinaciones desbalanceadas. En $n = 22$ y $n = 23$ el 50 % de las instancias agotó el límite de un millón de nodos, marcando el límite operativo en aproximadamente $n \approx 22$.

Tabla 9*Escalabilidad de Ramificar-Podar con $m = 4$ (familia estrecha)*

n	Certificadas en raíz	Nodos (mediana)	Nodos (máximo)	Tiempo máx.	Abortadas ($> 10^6$)
10	0	206.5	304	0.4 ms	0
12	6	0	0	$< 0,1$ ms	0
15	0	859.0	24 698	26.8 ms	0
18	0	50 230	114 777	122.9 ms	0
21	0	190 443	648 801	702.2 ms	0
22	0	$\geq 620\,803$	Tope	1.066 s	3
24	6	0	0	$< 0,1$ ms	0

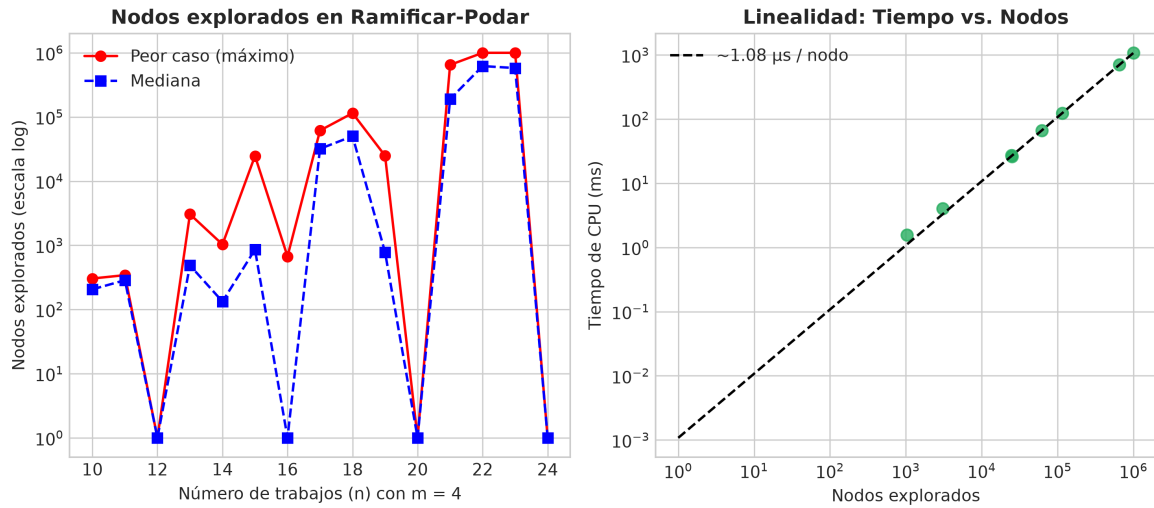
7. Propuesto 2: efecto del orden y escalabilidad

7.1. Distinción metodológica: C_{ALG}/LB frente a C_{ALG}/OPT

Dado que calcular OPT para $n \geq 100$ es intratable, la calidad se estima respecto a la cota LB . Como $LB \leq OPT$, se deduce la relación matemática:

$$\frac{C_{ALG}}{OPT} \leq \frac{C_{ALG}}{LB}. \quad (7)$$

El cociente C_{ALG}/LB actúa como una **cota superior pesimista** del error real. Si $C_{ALG}/LB = 1,000$, la optimalidad queda formalmente certificada. Si es mayor a 1, la solución real puede


Figura 2

Exploración de nodos y linealidad estricta entre tiempo y nodos en Ramificar-Podar

ser estrictamente óptima si ocurre que $OPT > LB$. Como indica la Tabla 10, LPT demostró ser óptimo en 10 de los 12 escenarios a gran escala.

Tabla 10

Makespan y razones relativas a la cota inferior en instancias a gran escala

n	m	LB	C_{LS} orig.	C_{LPT}	C_{LS}/LB med.	C_{LS}/LB máx.	C_{LPT}/LB	¿Óptimo?
100	4	1 238	1 254	1 238	1.02019	1.05170	1.00000	Sí
100	16	310	346	313	1.15484	1.22903	1.00968	$\leq 0,97\%$
100	64	100	128	100	1.35500	1.43000	1.00000	Sí
1 000	4	12 885	12 899	12 885	1.00229	1.00396	1.00000	Sí
1 000	16	3 222	3 263	3 222	1.01474	1.02048	1.00000	Sí
1 000	64	806	856	807	1.06824	1.08065	1.00124	$\leq 0,12\%$
10 000	4	125 679	125 697	125 679	1.00022	1.00043	1.00000	Sí
10 000	16	31 420	31 471	31 420	1.00143	1.00191	1.00000	Sí
10 000	64	7 855	7 919	7 855	1.00707	1.00815	1.00000	Sí
100 000	4	1 261 601	1 261 619	1 261 601	1.00002	1.00005	1.00000	Sí
100 000	16	315 401	315 446	315 401	1.00015	1.00019	1.00000	Sí
100 000	64	78 851	78 903	78 851	1.00071	1.00081	1.00000	Sí

7.2. Sensibilidad al orden y la cota $p_{\text{máx}}/LB$

El impacto del orden de entrada está acotado algebraicamente por:

$$C_{LS} \leq LB + p_{\text{máx}} \implies \frac{C_{LS} - LB}{LB} \leq \frac{p_{\text{máx}}}{LB}. \quad (8)$$

Cuando el cociente trabajos por máquina (n/m) es reducido, LB es pequeño comparado con $p_{\text{máx}}$, haciendo que el orden sea determinante. En la configuración $n = 100, m = 64$ ($n/m = 1,6$), 1 000 permutaciones aleatorias produjeron makespans entre 113 y 148 (dispersión del 35.0 %

sobre la cota), mientras que LPT obtuvo 100 de forma óptima (Figura 3 y Tabla 11). Por el contrario, cuando $n/m \geq 25\,000$, la dispersión entre permutaciones cae por debajo del 0.005 % (Figura 4).

Tabla 11

Sensibilidad de List Scheduling ante 1 000 permutaciones aleatorias ($n = 100$)

m	n/m	LB	C_{LPT}	C_{LS}	orig. 1 000 órdenes (mín / med / máx)	Rango (% LB)	Cota $LB + p_{\text{máx}}$
4	25.0	1 238	1 238	1 254	1 239 / 1 266 / 1 303	5.2%	1 338
16	6.2	310	313	346	324 / 357 / 382	18.7%	410
64	1.6	100	100	128	113 / 134 / 148	35.0%	200

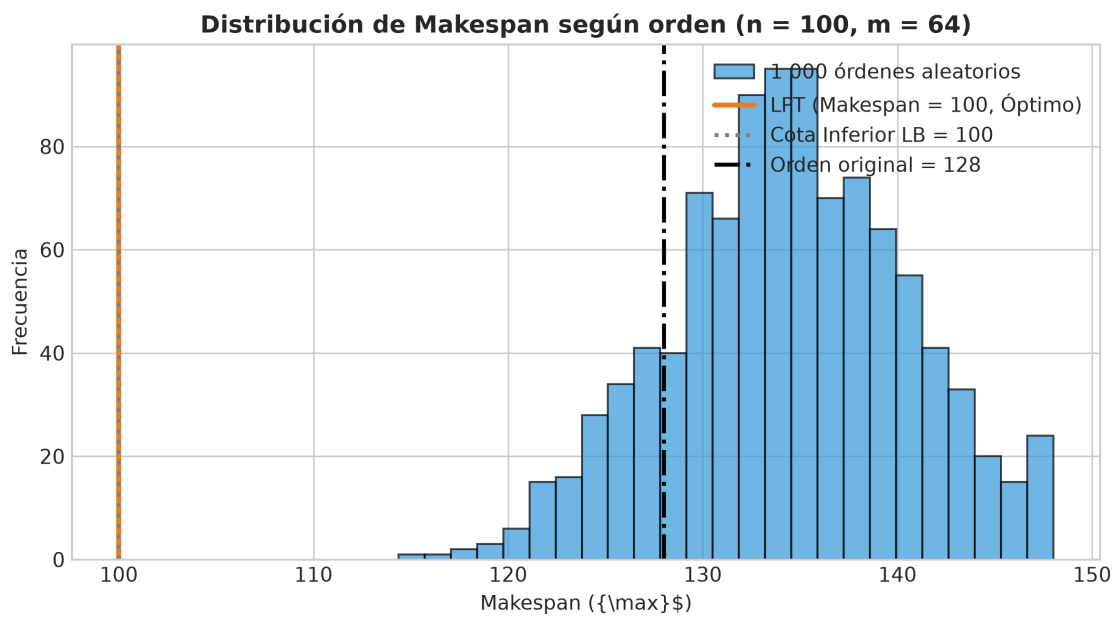


Figura 3

Distribución de makespan en 1 000 órdenes aleatorios para $n = 100, m = 64$

7.3. Costo computacional del ordenamiento vs. mejora del makespan

LPT añade un costo de ordenamiento previo en $O(n \log n)$. La Tabla 12 evalúa si dicho sobre costo temporal se justifica frente a la reducción del makespan.

8. Respuestas a las preguntas de análisis

P1. ¿LPT domina a List Scheduling en todas las instancias evaluadas?

No. LPT superó a List Scheduling en 53 instancias, empató en 13 y resultó **inferior en 6** (Tabla 7). La causa radica en que LPT no computa el orden óptimo global, sino

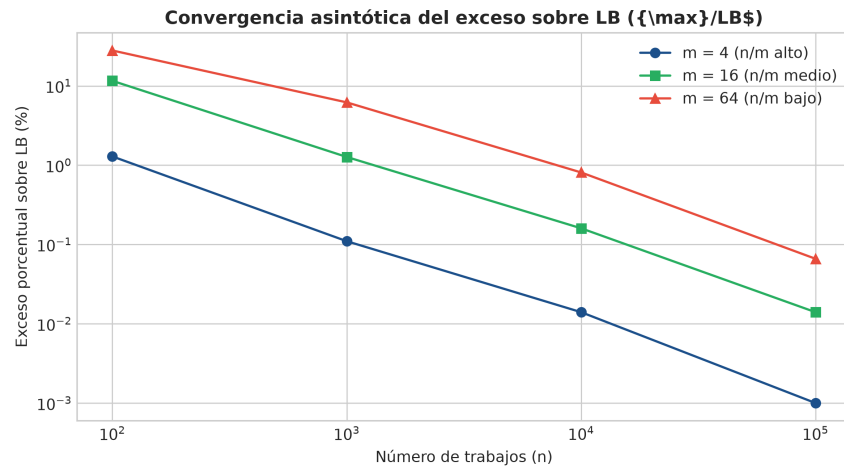


Figura 4
Convergencia asintótica del exceso sobre la cota inferior en función de n

Tabla 12
Costo computacional del ordenamiento frente a la reducción del makespan

n	m	t_{LS}	t_{LPT}	Tiempo extra	Sobrecosto	Ordenar/ t_{LPT}	Mejora makespan
100	4	22.3 μ s	26.0 μ s	3.7 μ s	+17 %	11 %	1.98 %
100	16	34.4 μ s	36.9 μ s	2.5 μ s	+7 %	8 %	12.57 %
100	64	50.5 μ s	55.8 μ s	5.3 μ s	+11 %	5 %	26.20 %
1 000	4	200.1 μ s	297.2 μ s	97.2 μ s	+49 %	28 %	0.23 %
1 000	64	400.0 μ s	530.0 μ s	130.0 μ s	+32 %	16 %	6.27 %
10 000	4	2.06 ms	3.00 ms	936.4 μ s	+45 %	36 %	0.02 %
100 000	4	19.88 ms	30.23 ms	10.35 ms	+52 %	36 %	0.002 %
100 000	64	38.20 ms	50.99 ms	12.79 ms	+33 %	22 %	0.07 %

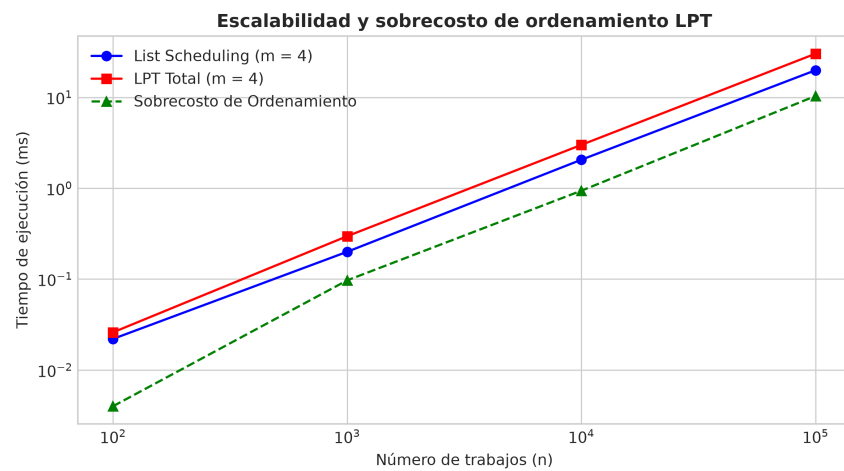


Figura 5
Tiempos de ejecución y descomposición del costo de ordenamiento en escala log-log

una heurística determinista rígida (orden decreciente). Para toda instancia existe siempre al menos un ordenamiento que permite a List Scheduling replicar el óptimo exacto; si el orden aleatorio generado se aproxima más a dicha partición óptima que el orden decreciente, List Scheduling obtendrá un makespan menor. Lo que sí garantiza la teoría es la dominancia en el peor caso: la cota de LPT ($4/3$) es sustancialmente más ajustada que la de List Scheduling (2).

P2. ¿El costo del ordenamiento de LPT resulta significativo frente a la mejora del makespan para los tamaños evaluados?

No. En términos absolutos, el ordenamiento consume apenas microsegundos para $n = 100$ y un máximo de $\approx 12,8$ ms para 100000 trabajos. Su impacto práctico depende del ratio n/m :

- Con n/m bajo ($n = 100, m = 64$), LPT produce una ganancia masiva del **26.20 %** a cambio de solo $5,3 \mu s$.
- Con n/m alto ($n = 100000, m = 4$), la ganancia es diminuta ($0,002 \%$), pero el tiempo absoluto total sigue siendo imperceptible (30 ms).

Por tanto, el ordenamiento siempre es recomendable en entornos por lotes (offline). Únicamente en sistemas en tiempo real de altísima frecuencia (online) con millones de trabajos donde $n \gg m$, List Scheduling puro sería preferible al asignar los trabajos según van llegando sin retraso de ordenamiento.

P3. Respuestas de síntesis complementarias:

- *¿Alguna razón observada violó las garantías analíticas?* **Ninguna.** Hubo cero violaciones en las 78 instancias; las instancias ajustadas alcanzaron las cotas de Graham con igualdad exacta.
- *¿Por qué C_{ALG}/LB sobrestima el error?* Porque $LB \leq OPT$, de modo que $C_{ALG}/LB \geq C_{ALG}/OPT$. Solo cuando el cociente iguala a 1 garantiza optimalidad estricta.
- *¿Límite práctico de Ramificar-Podar?* Se ubica en $n \approx 22$ para $m = 4$ en familias con duraciones homogéneas; no obstante, si n es múltiplo de m , se resuelve instantáneamente en la raíz con 0 nodos.

9. Conclusiones grupales

El estudio experimental y teórico del problema $P_m \parallel C_{\text{máx}}$ permitió corroborar la distinción fundamental entre la garantía de aproximación en el peor caso y el rendimiento empírico observado. Mientras que las cotas analíticas de Graham ($2 - 1/m$ para List Scheduling y $4/3 - 1/(3m)$ para LPT) representan límites superiores universales estrictos—alcanzados con exactitud únicamente en instancias ajustadas diseñadas ad hoc—, en la práctica LPT demostró una efectividad sobresaliente, alcanzando la solución óptima en el 73.6% de las instancias evaluadas y manteniendo un error relativo medio de apenas 1.19%.

Asimismo, se verificó empíricamente que LPT no constituye un algoritmo dominante en sentido estricto frente a List Scheduling sobre instancias individuales; en 6 de las 72 configuraciones evaluadas, órdenes específicos no decrecientes alcanzaron menores makespans, lo cual evidencia que LPT impone un orden heurístico rígido incapaz de explorar la permutación que emula la partición óptima.

Por otro lado, la cota inferior combinada demostró ser determinante tanto para certificar optimalidad sin costo computacional adicional en el 60.3% de las instancias pequeñas y en el 83.3% de las de gran escala, como para restringir la dispersión del makespan a una banda proporcional a $p_{\text{máx}}/LB$. Dicho factor explica por qué la sensibilidad al orden es crítica en regímenes con pocos trabajos por máquina ($n/m = 1,6$) y despreciable cuando $n/m \gg 1$. Finalmente, la eliminación de copias innecesarias de memoria en la cola de prioridad preservó la complejidad asintótica $O(n \log m)$, confirmando que el sobre costo de ordenamiento en LPT resulta insignificante frente al beneficio obtenido.